Name: Bhavya PS **Date:** 27/07/2026

Reg No: 192524017

1. CAESAR CIPHER

AIM

To implement the Caesar Cipher simple substitution technique using Python.

Algorithm

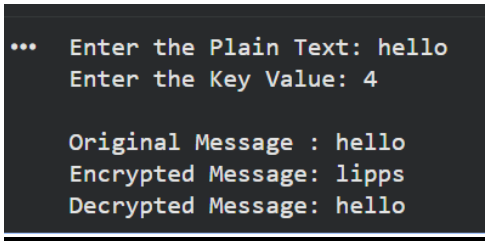
1. Start the program.
2. Take a plaintext message.
3. Choose a shift value.
4. Shift each alphabet character by the given key.
5. Display the encrypted text.
6. Shift back by the same key to decrypt.
7. Display the original message.
8. Stop.

Program

```
def encrypt(text, shift):
    result = ""

    for ch in text:
        if ch.isalpha():
            if ch.isupper():
                result += chr((ord(ch) - ord('A') + shift) % 26 + ord('A'))
            else:
                result += chr((ord(ch) - ord('a') + shift) % 26 + ord('a'))
        else:
            result += ch

    return result



message = input("Enter the Plain Text: ")
key = int(input("Enter the Key Value: "))
```

```
encrypted = encrypt(message, key)
decrypted = encrypt(encrypted, -key)

print("\nOriginal Message :", message)
print("Encrypted Message:", encrypted)
print("Decrypted Message:", decrypted)
```

The output:



RESULT:

Thus the implementation of Caesar cipher had been executed successfully.

2. MONOALPHABETIC CIPHER

AIM

To implement the Monoalphabetic Cipher using Python.

Algorithm

1. Start.
2. Define normal alphabets.
3. Define substituted alphabets.
4. Replace each plaintext letter with its corresponding substituted letter.
5. Display encrypted text.
6. Reverse the substitution for decryption.
7. Display decrypted text.
8. Stop.

The program:

```
plain_alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

cipher_alphabet = "QWERTYUIOPASDFGHJKLZXCVBNM"


plaintext = input("Enter the Plain Text: ").upper()


ciphertext = ""


for ch in plaintext:
    if ch.isalpha():
        index = plain_alphabet.index(ch)
        ciphertext += cipher_alphabet[index]
    else:
        ciphertext += ch


print("\nOriginal Alphabet :")
print(plain_alphabet)


print("\nSubstituted Alphabet :")
print(cipher_alphabet)


print("\nPlain Text  :", plaintext)
```

```
print("Cipher Text :", ciphertext)
```

the output:

```
Enter the Plain Text: hello

Original Alphabet :
ABCDEFGHIJKLMNOPQRSTUVWXYZ

Substituted Alphabet :
QWERTYUIOPASDFGHJKLZXCVBNM

Plain Text  : HELLO
Cipher Text : ITSSG
```

RESULT:

Thus the implementation of MONOALPHABETIC CIPHER had been executed successfully.

3. POLYALPHABETIC (VIGENERE) CIPHER

AIM

To implement the Polyalphabetic (Vigenère) Cipher using Python.

Algorithm

1. Start.
2. Take plaintext and keyword.
3. Repeat keyword until it matches message length.
4. Encrypt each letter using the keyword.
5. Display ciphertext.
6. Reverse the process for decryption.
7. Stop.

The program:

```
def encrypt(text, key):
    text = text.upper()
    key = key.upper()
    cipher = ""
    key_index = 0

    for ch in text:
        if ch.isalpha():
            shift = ord(key[key_index % len(key)]) - ord('A')
            cipher += chr((ord(ch) - ord('A') + shift) % 26 + ord('A'))
            key_index += 1
        else:
            cipher += ch

    return cipher



def decrypt(cipher, key):
    key = key.upper()
    plain = ""
    key_index = 0

    for ch in cipher:
        if ch.isalpha():
```

```
            shift = ord(key[key_index % len(key)]) - ord('A')
            plain += chr((ord(ch) - ord('A') - shift) % 26 + ord('A'))
            key_index += 1
        else:
            plain += ch

    return plain

plaintext = input("Enter the Plain Text: ").upper()
key = input("Enter the Keyword: ").upper()

ciphertext = encrypt(plaintext, key)
decryptedtext = decrypt(ciphertext, key)

print("\nPlain Text      :", plaintext)
print("Keyword         :", key)
print("Encrypted Text  :", ciphertext)
print("Decrypted Text  :", decryptedtext)
```

the output:

```
Enter the Plain Text: bhavya
Enter the Keyword: good

Plain Text      : BHAVYA
Keyword         : GOOD
Encrypted Text  : HVOYEO
Decrypted Text  : BHAVYA
```

RESULT:

Thus the implementation of POLYALPHABETIC (VIGENERE) CIPHER had been executed successfully.

4. HILL CIPHER

AIM

To implement the Hill Cipher using Python.

Algorithm

1. Start.
2. Choose a 2×2 key matrix.
3. Convert plaintext letters into numbers.
4. Multiply the key matrix with plaintext matrix.
5. Apply modulo 26.
6. Convert numbers back into letters.
7. Display encrypted text.
8. Stop.

The program:

```
key = [[3, 3],
       [2, 5]]


text = "HI"


p = [ord(text[0])-65, ord(text[1])-65]


c1 = (key[0][0]*p[0] + key[0][1]*p[1]) % 26
c2 = (key[1][0]*p[0] + key[1][1]*p[1]) % 26


cipher = chr(c1+65) + chr(c2+65)


print("Original :", text)
print("Encrypted:", cipher)
```

the output:

```
Original : HI
Encrypted: TC
```

RESULT:

Thus the implementation of HILL CIPHER had been executed successfully.

5. PLAYFAIR CIPHER

AIM

To implement the Playfair Cipher using Python.

Algorithm

1. Start.
2. Create a 5×5 matrix using the keyword.
3. Divide plaintext into letter pairs.
4. Encrypt each pair according to Playfair rules.
5. Display ciphertext.
6. Stop.

The program:

```
def generate_matrix(key):
    key = key.upper().replace("J", "I")
    matrix = []
    used = set()

    for ch in key:
        if ch.isalpha() and ch not in used:
            used.add(ch)
            matrix.append(ch)

    for ch in "ABCDEFGHIKLMNOPQRSTUVWXYZ":
        if ch not in used:
            used.add(ch)
            matrix.append(ch)

    return [matrix[i:i+5] for i in range(0, 25, 5)]



def find_position(matrix, ch):
    if ch == "J":
        ch = "I"

    for i in range(5):
        for j in range(5):
            if matrix[i][j] == ch:
                return i, j
```

```
def prepare_text(text):
    text = text.upper().replace("J", "I")
    text = "".join(ch for ch in text if ch.isalpha())

    result = ""
    i = 0

    while i < len(text):
        a = text[i]

        if i + 1 < len(text):
            b = text[i + 1]
            if a == b:
                result += a + "X"
                i += 1
            else:
                result += a + b
                i += 2
        else:
            result += a + "X"
            i += 1

    return result



def encrypt(text, matrix):
    text = prepare_text(text)
    cipher = ""

    for i in range(0, len(text), 2):
        a = text[i]
        b = text[i + 1]

        r1, c1 = find_position(matrix, a)
        r2, c2 = find_position(matrix, b)

        if r1 == r2:
            cipher += matrix[r1][(c1 + 1) % 5]
            cipher += matrix[r2][(c2 + 1) % 5]
```

```
        elif c1 == c2:
            cipher += matrix[(r1 + 1) % 5][c1]
            cipher += matrix[(r2 + 1) % 5][c2]

        else:
            cipher += matrix[r1][c2]
            cipher += matrix[r2][c1]

    return cipher


key = "MONARCHY"
plaintext = "HELLO"

matrix = generate_matrix(key)

print("Playfair Matrix:")
for row in matrix:
    print(" ".join(row))

cipher = encrypt(plaintext, matrix)

print("\nPlaintext :", plaintext)
print("Encrypted :", cipher)
```

the output:

```
Playfair Matrix:
M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X Z

Plaintext : HELLO
Encrypted : CFSUPM
```

RESULT:

Thus the implementation of PLAYFAIR CIPHER had been executed successfully.

6. RAIL FENCE CIPHER

AIM

To implement the Rail Fence Transposition Cipher using Python.

Algorithm

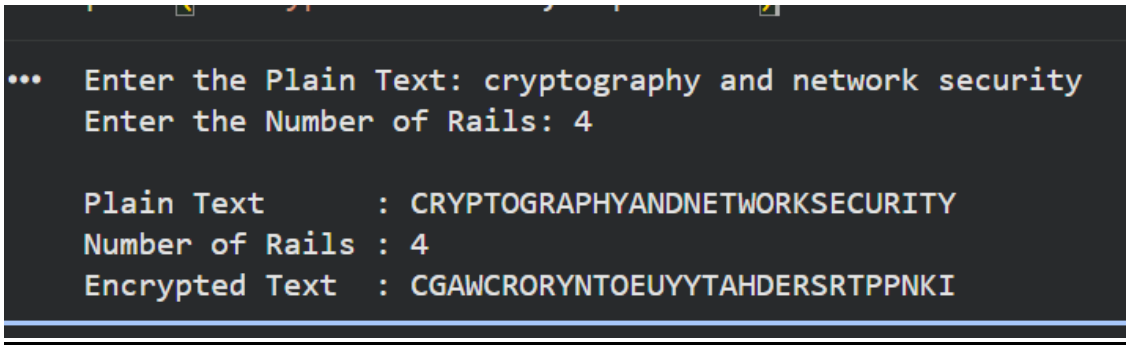
1. Start.
2. Choose the number of rails.
3. Write the plaintext in a zigzag pattern.
4. Read row by row.
5. Display ciphertext.
6. Stop.

The program:

```
def rail_fence_encrypt(text, rails):
    fence = ['' for _ in range(rails)]

    rail = 0
    direction = 1

    for ch in text:
        fence[rail] += ch

        if rail == 0:
            direction = 1
        elif rail == rails - 1:
            direction = -1

        rail += direction

    return ''.join(fence)



plaintext = input("Enter the Plain Text: ").upper().replace(" ", "")
rails = int(input("Enter the Number of Rails: "))

ciphertext = rail_fence_encrypt(plaintext, rails)

print("\nPlain Text      :", plaintext)
print("Number of Rails :", rails)
```

```
print("Encrypted Text  :", ciphertext)
```

the output:



RESULT:

Thus the implementation of RAIL FENCE CIPHER had been executed successfully.

7. COLUMNAR TRANSPOSITION CIPHER

AIM

To implement the Columnar Transposition Cipher using Python.

Algorithm

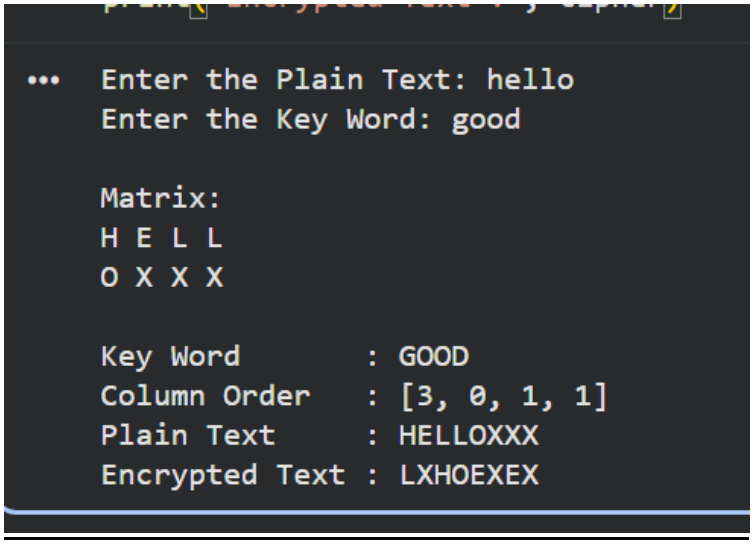
1. Start.
2. Write plaintext row-wise in a table.
3. Arrange columns according to the key.
4. Read the columns in sorted key order.
5. Display ciphertext.
6. Stop.

The program:

```
import math


plaintext = input("Enter the Plain Text: ").upper().replace(" ", "")
key = input("Enter the Key Word: ").upper()


cols = len(key)
rows = math.ceil(len(plaintext) / cols)
plaintext += "X" * (rows * cols - len(plaintext))
matrix = []
k = 0


for i in range(rows):
    row = []
    for j in range(cols):
        row.append(plaintext[k])
        k += 1
    matrix.append(row)


sorted_key = sorted(list(key))
order = []


for ch in sorted_key:
    order.append(key.index(ch))
cipher = ""


for col in order:
```

```
    for row in range(rows):
        cipher += matrix[row][col]
print("\nMatrix:")
for row in matrix:
    print(" ".join(row))

print("\nKey Word       :", key)
print("Column Order   :", order)
print("Plain Text     :", plaintext)
print("Encrypted Text :", cipher)
```

the output:



RESULT:

Thus the implementation of COLUMNAR TRANSPOSITION CIPHER had been executed successfully.